

Self-HTR: A Novel Self-Supervised Handwritten Text Recognition Framework Using Generative Adversarial Networks - Supplementary Materials

No Author Given

No Institute Given

1 HTR Model Optimization

Our proposed self-HTR framework uses the CNN-BLSTM architecture proposed in [12]. However, while our work is applied to word recognition, the CNN-BLSTM was proposed for line text recognition. As we found the model’s performance on word recognition to be lower compared to that reported for line text recognition in [12], additional experiments were conducted to further optimize the HTR model architecture and supervised training process for word recognition. This section details these experiments and reports their results.

1.1 Experimental Setup

To optimize the HTR architecture, we considered recommendations proposed in [13]. Specifically, performance of the model architecture using column-wise concatenation and max-pooling of the CNN’s extracted feature maps were compared. Column-wise max pooling would introduce translation invariance w.r.t. the characters’ vertical position in the images. Moreover, it reduces computational costs by reducing the size of the feature vectors.

Next to optimizing the HTR architecture, the supervised training process was optimized as well. To do so, the CTC loss originally proposed for the CNN-BLSTM was compared with commonly used cross-entropy (CE) loss. The probabilistic nature of the CE loss specifically makes it well-suited for sequential word recognition. Moreover, considering the applicability of a pretrained HTR network to self-HTR, the CE loss does not require an extra blank symbol in the vocabulary and is therefore more compatible with pretrained HTG generator.

Given the adaptation of the architecture and the two loss functions, models were trained in four experimental conditions with either column-wise concatenation or max pooling, and either the CTC or CE loss. In each setting, the Adam optimizer was applied with a learning rate of 0.0001. The learning rate was reduced by 20% when the validation loss did not increase for at least 5 epochs. The batch size was set to 16 samples.

1.2 Evaluation

The supervised HTR models were trained on the IAM-HTR-IV training set and evaluated on the HTR-IAM-IV and HTR-IAM-OOV validation and test sets with

the character error rate (CER) and word error rate (WER). The predictions were obtained with greedy decoding. Early stopping was applied on the IAM-HTR-IV validation loss with a patience of 10 epochs. To prevent tuning on the test set, the best model architecture was selected based on the validation results.

1.3 Results

The results on the IAM-HTR validation sets as presented in Table 1 shows column-wise max pooling in combination with the CE loss resulted in the best supervised HTR model performance. Overall, it can be observed column-wise max pooling led to an increased performance for both losses and on both the IV and OOV validation sets. This strongly suggests HTR models benefit from feature extraction where the characters’ horizontal position is emphasized more. Considering the loss functions, the results showed an overall increased model performance for the CE loss compared to the CTC loss, with the WER being prioritized over the CER. Worth noting is that all models perform considerably worse on the OOV validation set compared to the IV validation set, showing the HTR model struggles to generalize to unseen words.

Given these results, self-HTR used column-wise max pooling in its HTR architecture, and all pretrained supervised models were trained with the CE loss.

Table 1: Validation CER and WER on IAM-HTR per supervised HTR model configuration.

Method	IV Validation		OOV Validation	
	CER (%)	WER (%)	CER (%)	WER (%)
CE + concatenation	34.70	55.42	68.89	99.42
CE + max pooling	15.14	32.91	45.57	92.84
CTC + concatenation	31.28	58.22	65.54	99.88
CTC + max pooling	20.65	51.86	43.66	95.84

2 HTG Model Selection

In order for self-HTR to be able to effectively learn, the synthetic images need to be of good quality. As such, this section details the selection of the HTG generator for self-HTR.

GANwriting was trained on the IAM-GEN training set and evaluated with the validation set at varying epochs. Specifically, the model was trained for 3000 epochs and evaluated with the Frechet Inception Distance (FID) every 250 epochs starting from epoch 1750. This was determined based on when the generator loss indicated convergence (see Figure 1).

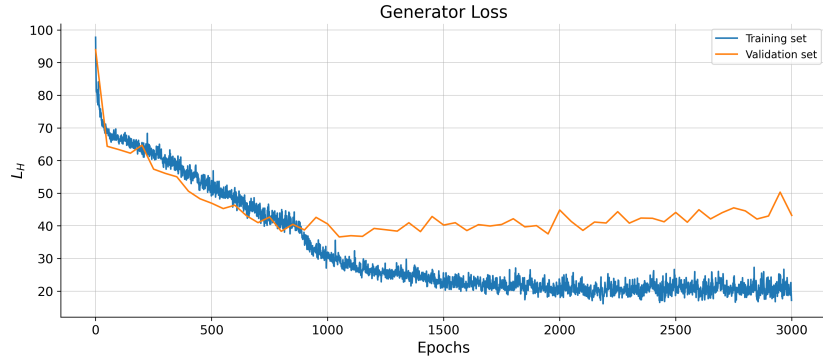


Fig. 1: GANwriting generator loss curve

The FID, computed as in Equation 1, is the Fréchet distance between the normal distributions with means and covariance matrices μ_r , Σ_r for real images, and μ_g , Σ_g for generated images. These are extracted from the feature maps of the Inception-V3 network pretrained on ImageNet [5]. While there is discourse about the appropriateness of the FID for images with handwriting, the metric is the only one consistently used in HTG. Hence, the final model was selected based on the lowest FID score, indicating the highest image quality.

$$d(\mathcal{N}(\mu_r, \Sigma_r), \mathcal{N}(\mu_g, \Sigma_g))^2 = \|\mu_g - \mu_r\|_2^2 + \text{Tr}(\Sigma_g + \Sigma_r - 2(\Sigma_g \Sigma_r)^{\frac{1}{2}}) \quad (1)$$

Table 2 shows the FID scores across epochs, starting from epoch 1750. These results show that the generator trained for 3000 epochs produced the best quality images (FID = 30.36), and the worst at epoch 1750 (FID = 46.12). A set of 16 synthetic images with their real counterparts are shown across epochs in Figure 2. It can be observed the model overall struggles with accurately imitating the writing styles, specifically slanted styles. Moreover, characters are on different scales compared to the real images, and not all letters are legible (e.g., ‘w’ and ‘ng’). Based on the FID scores, being a more general and objective measure than image comparison, the generator model trained for 3000 epochs was selected as the best-performing generator.

Table 2: FID scores across epochs of GANwriting on IAM-GEN validation data.

Epoch	1750	2000	2250	2500	2750	3000
FID	46.12	40.96	42.93	38.08	30.65	30.36

To further assess synthetic image quality, the best-performing supervised HTR model was trained on the recreated IAM-HTR dataset and evaluated on

both its original and recreated validation sets. This evaluation method is based on [11], where similar performance on original test data implies high-quality synthetic images. Table 3 shows these results, indicating that the HTR model does not generalize well to the original IAM-HTR validation data, suggesting a strong discrepancy between the original and synthetic images. This motivated the use of style-invariant losses for self-HTR.



Fig. 2: Generated images across epochs 1750 to 3000 in steps of 250 epochs.

3 A Style-Invariant Siamese Network

This section details the methodology and results of the Siamese Network used for the Siamese Network loss. This approach was inspired by [2], where a Siamese Network was trained for word spotting. They showed that image representations learned by a Siamese Network could be used to rank the similarity of pairs of word images. This was applied to self-HTR by training a Siamese Network which extracts word image representations such that real and synthetic images are similar if they contain the same text and dissimilar otherwise, thereby circumventing stylistic differences.

Table 3: Original and recreated IAM-HTR validation results of the best-performing supervised HTR model trained on the recreated IAM-HTR dataset.

Validation Data	IV		OOV	
	CER (%)	WER (%)	CER (%)	WER (%)
Recreated IAM-HTR	0.67	2.91	5.25	21.59
Original IAM-HTR	53.43	82.82	57.21	95.50

3.1 Dataset

To ensure independence between the Siamese Network and self-HTR’s network, the Siamese Network was trained on the IAM-GEN-SIA dataset. This was obtained by merging and refactoring the partitions of IAM-GEN into a training (70%, 21907 samples), validation (15%, 4565 samples), and test set (15%, 4824) such that each data split contained all writing styles. The preprocessing was consistent with the HTR and HTG pipelines to ensure compatibility with these models.

In order for the Siamese Network to learn feature representations such that those of real and synthetic images with the same text are similar and dissimilar otherwise, we created two similar and two dissimilar image pairs for each original, real-life image in IAM-GEN-SIA. The pairs were created by synthesizing images with GANwriting’s pretrained generator. Similar (positive) pairs contained the same text, with one of the synthetic images having the same writer style and the other a randomly selected style. Dissimilar (negative) pairs were in the same writer style but with different texts—one randomly generated and one edited from the real image’s text. In addition, there was a 25% chance of duplicating characters in both these text labels. In this manner, negative pairs simulate training conditions of self-HTR. Examples of the image pairs are shown in Figure 3. The following details the methods used to obtain the negative pairs’ text labels.

Random generation Text labels were generated by randomly sampling a random number of $N = \{0, 1, 2, \dots, 10\}$ characters from a vocabulary of 52 alphabetic characters (upper and lower cases), weighted by their frequencies in the IAM-GEN-SIA training set.

Label edits The text of the synthetic images in 50% of the dissimilar image pairs was obtained by modifying the text labels of the corresponding real image. Specifically, three consecutive character edits were applied. These were one of random insertion, deletion, swapping, and replacement. For the random insertion and replacement operations, characters were randomly sampled from the vocabulary, weighted by the IAM-GEN-SIA training set character frequencies.

Duplication Randomly generated and edited text labels had 25% chance duplication would be applied. If applied, one or two characters in the label were randomly selected and duplicated from 2 up to 5 times.

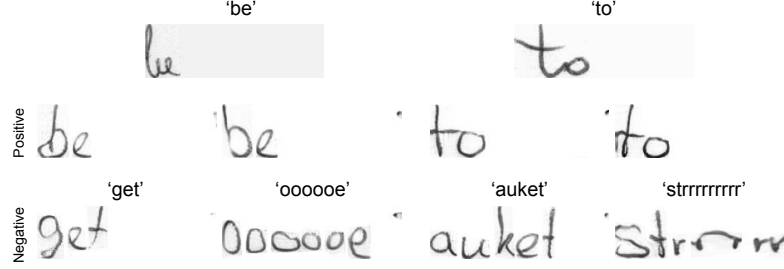


Fig. 3: Samples of the synthetic images for the positive and negative pairs with their real counterparts.

3.2 Network Architecture

Following [2], the Siamese Network learns a feature space via explicit distance computation. The Siamese Network architecture consists of two identical branches whose weights are shared. An overview is shown in Figure 4. Based on initial experimentation, feature maps were extracted using a CNN with the ResNet34 architecture [7] pretrained on ImageNet. These were then reduced via adaptive average pooling. Two dense layers with 256 neurons each generate the final feature representations, using ReLU activation and batch normalization in the first layer for regularization.

3.3 Implementation Details

The Siamese Network was trained with the contrastive loss function depicted in Equation 2, where $D(x_1, x_2)$ is the Euclidean distance between the feature vectors of input images x_1 and x_2 with label y , and $m = 1.0$. The left term of the loss then penalizes large distances between similar pairs, and the right term penalizes small distances between dissimilar pairs.

$$L_{contr}(\mathbf{x}_1, \mathbf{x}_2, y) = y \cdot D(x_1, x_2)^2 + (1 - y)(\max(0, m - D(x_1, x_2)))^2 \quad (2)$$

The network was trained with the RMSprop optimizer and an initial learning rate of 0.0001. The learning rate was decreased by a factor of 0.1 when the validation loss did not decrease for more than two epochs. The batch size was set to 16.

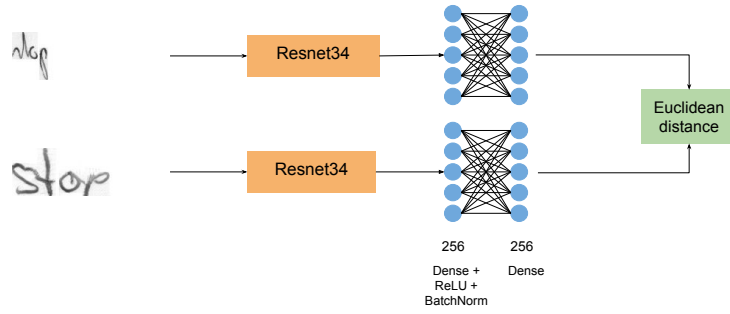


Fig. 4: Siamese Network architecture

3.4 Evaluation

The Siamese Network was trained on the IAM-GEN-SIA dataset and evaluated on its validation and test set. Early stopping was applied with a patience of five epochs to prevent overfitting. To evaluate the model's performance, the Euclidean distances between the similar and dissimilar pairs were computed across batches. A two-tailed t-test was performed on the test set to test for statistical significance.

3.5 Results

Table 4 shows the results of the Siamese Network on the IAM-GEN-SIA validation and test sets. Both partitions have a clear difference in distance between the similar and dissimilar pairs with minimal variance, suggesting appropriate feature representations were learned. Moreover, the results are consistent among the validation and test sets, indicating good generalization to unseen data. Loss curves (Figure 5) show minimal overfitting. A two-tailed t-test on the test results indicated that the distance between similar images was significantly smaller than the distance between dissimilar images ($t(2282) = 161.024$, $p < 0.005$).

Table 4: Siamese Network results on the IAM-GEN-SIA dataset.

	Euclidean distance validation set	Euclidean distance test set
Similar images	0.105 ± 0.069	0.107 ± 0.070
Dissimilar images	0.541 ± 0.054	0.539 ± 0.057
Difference	0.435 ± 0.085	0.432 ± 0.089

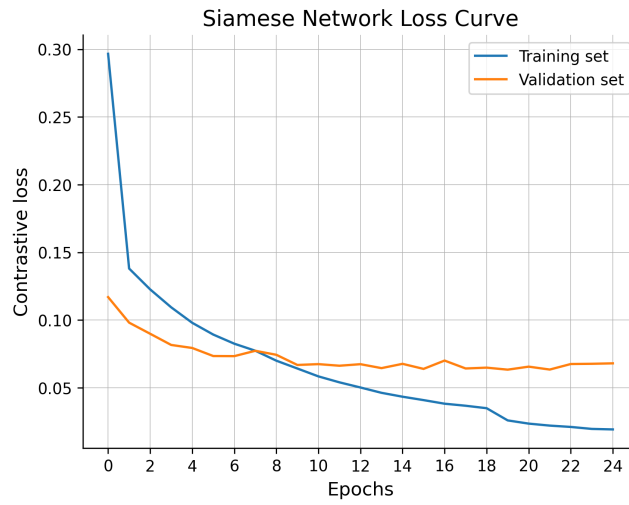


Fig. 5: Training and validation loss curves of the Siamese Network on IAM-GEN-SIA.

4 Loss Curves Self-HTR on Original IAM-HTR

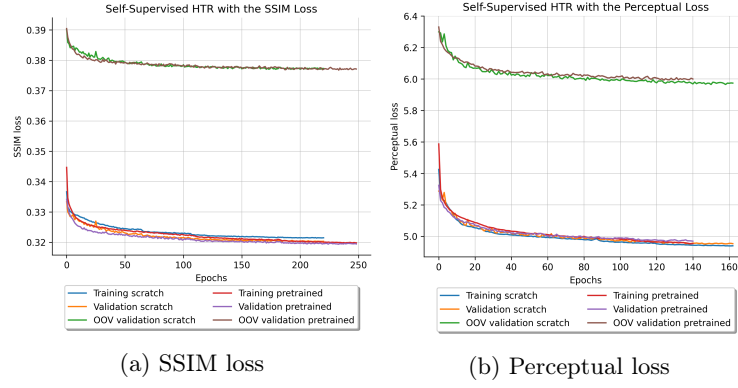


Fig. 6: Loss curves of self-supervised HTR trained with the SSIM and perceptual losses on original IAM-HTR data.

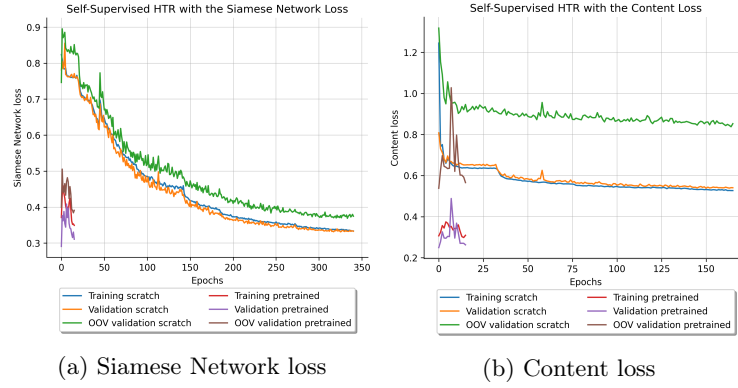


Fig. 7: Loss curves of self-supervised HTR trained with the Siamese Network and content losses on original IAM-HTR data.

5 Self-Supervised Handwritten Character Recognition

In this section we present the results of preliminary experiments of image-based self-supervised handwritten character recognition (self-HCR) on the MNIST dataset. Through these experiments, we explored the proposed framework’s feasibility and implementation. All models presented here were implemented with TensorFlow [1]. The code is publicly available at <https://github.com/Lisa-dk/self-supervised-mnist.git>.

5.1 Dataset

The MNIST dataset [9] is a widely used benchmark for handwritten digit classification, known for its clean images which makes it ideal for testing novel frameworks. It consists of 70,000 28 x 28 grayscale images of uniformly distributed handwritten digits 0 to 9. The images were rescaled to [0,1]. Example images are shown in Figure 8. The standard partitions of the dataset were used, consisting 60,000 training and 10,000 test images [9]. The training set was split in half into MNIST-HCR and MNIST-GEN for training HCR and generative models, respectively. The test set was reserved for MNIST-HCR.

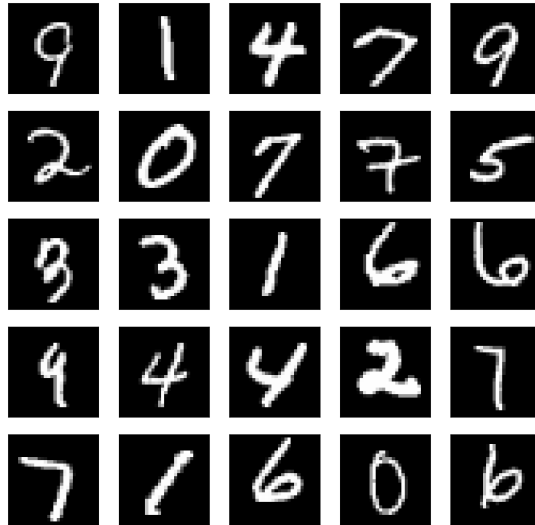


Fig. 8: Example images from the MNIST dataset.

5.2 Model Architectures

Handwritten Character Recognition For integrating the HCR and handwritten character generation (HCG) models in self-supervised HCR, a simple

lightweight HCR architecture was preferred. Hence, we designed a small CNN network for HCR that, although not state-of-the-art, could achieve a good performance. As shown in Figure 9, it consists of three convolutional layers with 64, 64, and 128 channels, each using 3×3 kernels. Strides of 2 were applied in the first two layers and 1 in the third. Leaky ReLU activation was used throughout, with dropout (0.2) applied after the first two layers to prevent overfitting. A final classification layer with 10 neurons using the Softmax activation output the class probabilities for each of the digits.

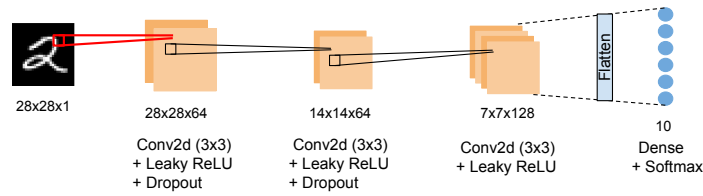


Fig. 9: The HCR architecture, using three convolutional layers for feature extraction and a dense layer for digit classification.

Handwritten Character Generation The Deep Convolutional GAN (DCGAN) [3] is a generative architecture that has been shown to perform well for handwritten character generation (HCG) on MNIST. It uses convolutions for up-sampling and feature extraction, which allows for higher image resolution and improves training stability. Due to its simplicity, DCGAN facilitates integration with the HCR architecture for the self-supervised framework. However, it has to be conditioned on the digit classes. We adapted the Keras implementation [4] which follows [10] to TensorFlow for compatibility with the HCR architecture. An overview of the conditional DCGAN is shown in Figure 10. The generator G synthesizes images from random noise conditioned on digit labels $\mathbf{y}$, while the discriminator D classifies images as real or synthetic given their labels. The following describes their architectures and conditioning in more detail.

The generator model’s architecture (Figure 11) is conditioned by concatenating one-hot encoded labels with 128-dimensional Gaussian noise. A dense layer reshapes this into a $7 \times 7 \times 128$ image, followed by two transposed convolutional layers (128 channels, 4×4 kernels, stride 2) and a final one-channel convolution (7×7 kernel, stride 2), producing a $28 \times 28 \times 1$ image. LeakyReLU was applied after each layer, except for the final Sigmoid-activated layer.

The discriminator model’s architecture is shown in Figure 12. The model is conditioned on class labels by duplicating and reshaping them into $28 \times 28 \times 10$ images, and concatenating with input images to form $28 \times 28 \times 11$ tensors. A CNN with two convolutional layers (64 and 128 channels, 3×3 kernels, stride 2) ex-

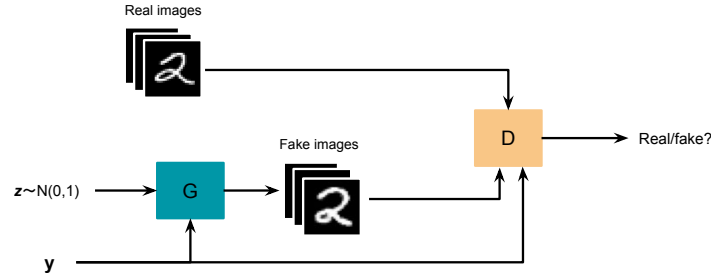


Fig. 10: Overview of [4]’s conditional DCGAN architecture. A generator G produces synthetic images from randomly sampled noise z with the given digits y . A discriminator predicts whether the images of the given digits are real or not.

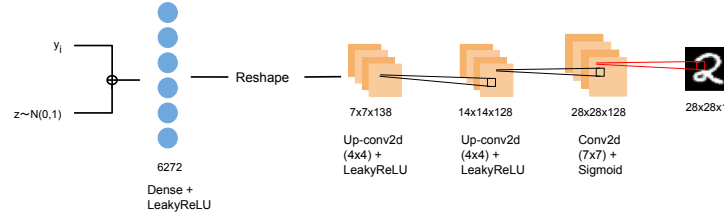


Fig. 11: The DCGAN’s generator architecture uses one dense and three up-sampling layers to produce synthetic images. It was conditioned on the digit classes by embedding the randomly sampled noise z with a label y_i .

tracts feature maps, which were reduced to a 128-dimensional feature vector with global max pooling. The Leaky ReLU activation function was applied after each convolutional layer, as well as dropout with a probability of 0.2 to prevent overfitting.

Self-Supervised Handwritten Character Recognition To integrate the HCR and HCG models into the self-supervised HCR framework, the generator was pretrained and kept fixed during self-supervised training to prevent training instability. A schematic overview of the framework is shown in Figure 13.

Compatibility between the generator and HCR model was ensured by applying identical image preprocessing and label encoding. To enable comparison of the synthetic and input images in a loss function, their pixel values were rescaled to the same range. Additionally, using one-hot encoded labels allowed direct input of HCR-predicted class probabilities into the generator, avoiding extra decoding steps and preserving confidence information. Using identical image preprocessing and one-hot encoded labels as described minimized the framework’s computational costs.

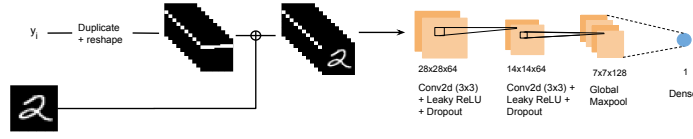


Fig. 12: The DCGAN’s discriminator architecture using a two-layer CNN. It was conditioned on the digit classes by concatenating the input image with the image representation of its label y_i .

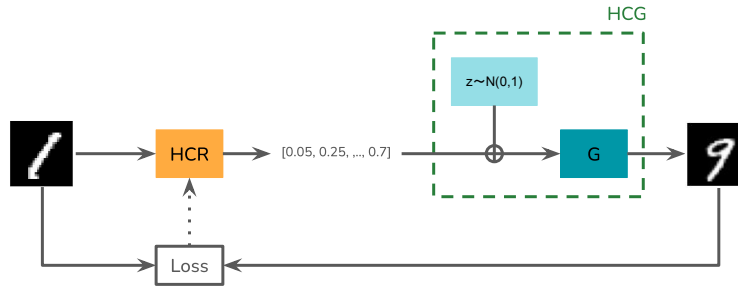


Fig. 13: A schematic overview of image-based self-supervised HCR. The generator G is pretrained.

5.3 Experimental Setup

Handwritten Character Generation To obtain an optimal HCG model for self-HCR, experiments were conducted with data augmentation where the DCGAN was trained in a non-augmented setting with only non-augmented images and an augmented setting with both non-augmented and augmented images. Each training image was augmented once with a random rotation and shearing, in both x and y axes, within the range of -10 to 10 degrees. Figure 14 shows a comparison between non-augmented and augmented images.

In both settings, the DCGAN was trained end-to-end with the adversarial loss. In each epoch, first the discriminator was updated and then the generator. The models were trained using the Adam optimizer with a learning rate of 0.0003. The batch size was set to 64.

Self-Supervised Handwritten Character Recognition For self-HCR, experiments were conducted with three loss functions. Framing self-HCR as an image reconstruction problem where the loss should capture well differences in character shapes, we considered the Mean Squared Error (MSE) and Binary Cross-Entropy (BCE) losses, inspired by [8]. These are computed as in Equations 3 and 4, where H and W are the image height and width, and $\hat{y}$ and y are the synthetic and input images, respectively.

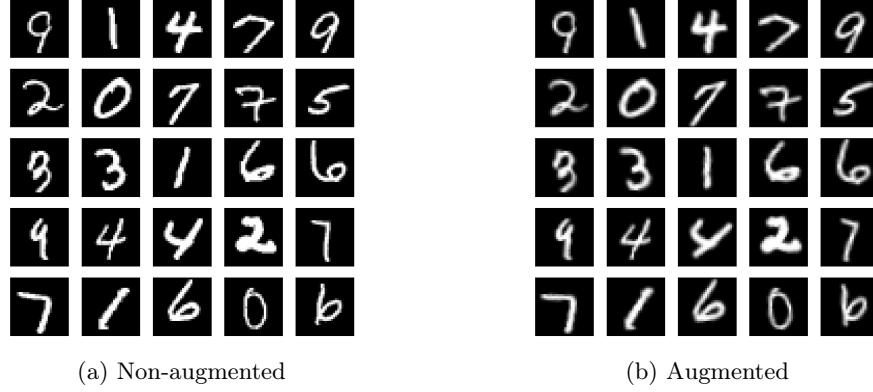


Fig. 14: Non-augmented (a) and augmented (b) samples of MNIST.

$$L_{MSE}(\hat{y}, y) = \frac{1}{HW} \|y - \hat{y}\|_2^2 \quad (3)$$

$$L_{BCE}(\hat{y}, y) = -\frac{1}{HW} \sum_{i=1}^W \sum_{j=1}^H [y_{i,j} \log(\hat{y}_{i,j}) + (1 - y_{i,j}) \cdot \log(1 - \hat{y}_{i,j})] \quad (4)$$

As a third loss function, the perceptual loss was used, which showed good performance for image generation tasks [6]. This loss (see Equation 5) operates on a higher level by measuring the difference between pixels of feature maps extracted from the activations of the j th layer of the VGG-16 network pretrained on ImageNet. To select the optimal layer for feature extraction, we trained k -nearest neighbor classifiers on feature maps from various convolutional layers (4th, 7th, 10th, and 13th) on the MNIST-HCR dataset with 5-fold cross-validation. The results in Table 5 show that the extracted feature maps at the 7th convolution resulted in the highest accuracy ($98.97\% \pm 0.11\%$) and were thus used to compute the perceptual loss.

$$L_p(\hat{y}, y) = \frac{1}{C_j H_j W_j} \|\Phi_j(\hat{y}) - \Phi_j(y)\|_2^2 \quad (5)$$

All self-HCR models were trained with the Adam optimizer and an initial learning rate of 0.001. A multi-step learning rate scheduler was used that decreased the learning rate to 0.0003, 0.0001, and 0.00001 after 1000, 3000, and 5000 iterations, respectively. The batch size was set to 256.

The self-supervised HCR models were trained on the MNIST-HCR dataset with 5-fold cross-validation. Early stopping was applied with a patience of 5 epochs to prevent overfitting. The models' test performances were obtained by taking the mean of the predicted class probabilities from the models resulting from each k -fold. Following the supervised setting, the models were evaluated with accuracy, and on test data the confusion matrix as well.

Table 5: Results of kNN classifiers trained with 5-fold cross-validation on the MNIST-HCR dataset. The models were trained on the activations of the feature maps extracted at the n -th convolution with the VGG-16 network pretrained on ImageNet.

N th convolution	Validation accuracy (%)
4th	97.80 ± 0.17
7th	98.97 ± 0.11
10th	98.42 ± 0.10
13th	93.90 ± 0.19

5.4 Evaluation

Handwritten Character Generation To train the DCGAN, 90% of the MNIST-GEN dataset was used for training (27,000 samples) and 10% for validation (3,000 samples). The number of training epochs for the models was based on the training and validation losses and the synthetic image quality. The synthetic image quality was evaluated visually by generating 10 images per digit class every 5 epochs. Additionally, the Fréchet inception distance (FID) was used for quantitative evaluation on the MNIST-GEN validation set. Because calculating the FID is computationally expensive, this was only computed from when the losses started to converge and when the synthetic images no longer appeared to improve visually.

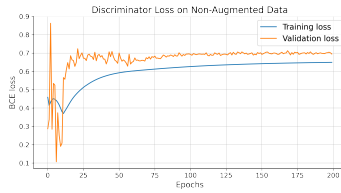
Self-supervised Handwritten Character Recognition The self-HCR models were trained on MNIST-HCR with 5-fold cross-validation. To prevent overfitting, early stopping was applied with a patience of 5 epochs. The models’ test performances were obtained by taking the mean of the predicted class probabilities from the models resulting from each k -fold. Since there were no heavy class imbalances, the models were evaluated with accuracy. The confusion matrix was computed on the test set to gain better insight into the model’s performance. Additionally, self-HCR’s model performance was compared with the supervised HCR performance on the test set. Except for a static learning rate of 0.001, the supervised HCR model was trained in the same manner as self-HCR.

5.5 Results

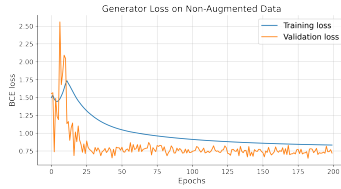
Handwritten Character Generation The DCGAN was trained with MNIST-HCG on only non-augmented images in a non-augmented setting and on both non-augmented and augmented images in an augmented setting. They were trained for 200 epochs since the losses in the non-augmented setting had reached convergence here. (see Figure 15). The FID was computed every 25 epochs, starting from the 150th epoch, as the loss curves in the non-augmented setting started

to converge here and no clear visual improvements of the synthetic images could be observed.

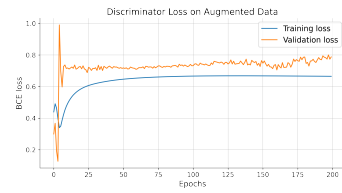
Overall, the DCGAN in the augmented setting converged faster and showed greater stability compared to that in the non-augmented setting when regarding the loss curves. However, showed instability from epoch 150 in the validation losses, especially in the discriminator’s validation loss. Figures 17 and 18 show a set of synthetic images across epochs 150, 175, and 200 for the non-augmented and augmented settings, respectively. Common artifacts such as distorted digits or extra ink blobs can be observed in both settings. Table 6 shows the FID for DCGAN models in both settings, where a lower FID indicated increased image quality. We can observe an improved image quality at epoch 200 compared to the earlier epochs for both settings as well as an overall improved synthetic image quality in the non-augmented setting compared to the augmented setting. Since the FID was lowest at epoch 200 in the non-augmented setting, this generator model was selected as the best-performing DCGAN and used in self-HCR.



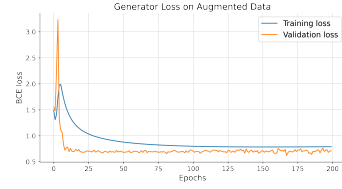
(a) Discriminator loss



(b) Generator loss



(a) Discriminator loss



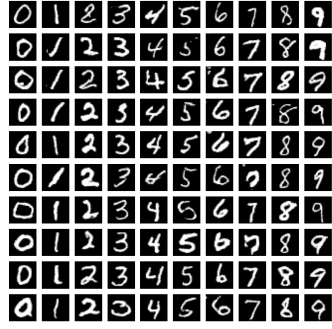
(b) Generator loss

Fig. 15: DCGAN loss curves on the non-augmented MNIST-GEN dataset.

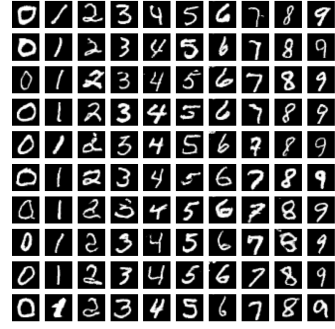
Fig. 16: DCGAN loss curves on the augmented MNIST-GEN dataset.

Table 6: FID computed between real and synthetic images on MNIST-GEN validation set.

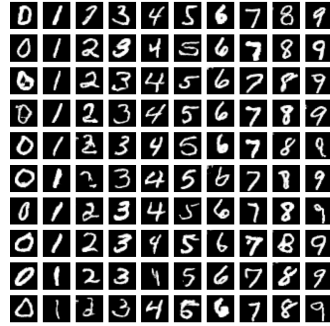
Epoch	FID Non-augmented	FID Augmented
150	12.286	12.317
175	12.034	12.288
200	11.936	12.137



(a) Epoch 150

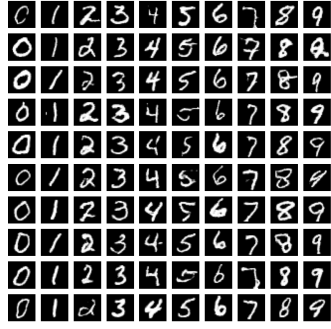


(b) Epoch 175

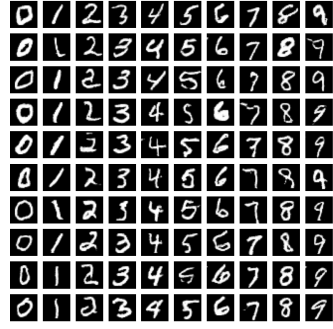


(c) Epoch 200

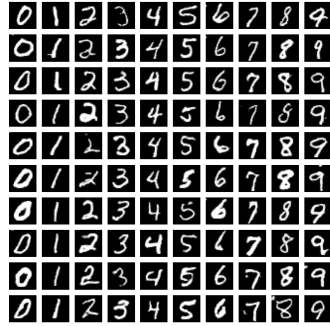
Fig. 17: Synthetic images of the DCGAN trained on non-augmented MNIST-GEN images after 150 (a), 175 (b), and 200 epochs (c)



(a) Epoch 150



(b) Epoch 175



(c) Epoch 200

Fig. 18: Synthetic images of the DCGAN trained on augmented MNIST-GEN images after 150 (a), 175 (b), and 200 epochs (c)

Table 7: Validation and test accuracy across loss functions for self-supervised HCR models trained on MNIST-HCR with 5-fold cross-validation.

Approach	Loss function	Validation accuracy (%)	Test accuracy (%)
Self-supervised	MSE	63.11 ± 7.14	66.87
	BCE	60.61 ± 1.95	62.44
	Perceptual	89.61 ± 1.29	91.99
Supervised	CE	97.97 ± 0.10	98.82

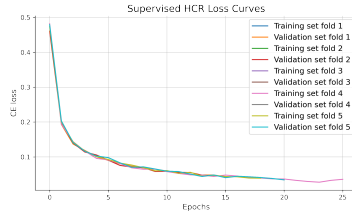


Fig. 19: Loss curves of the supervised HCR model for 5-fold cross-validation.

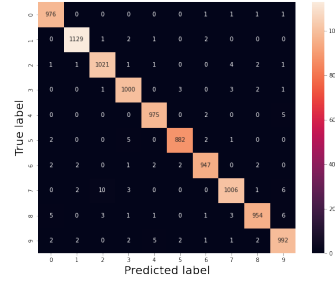


Fig. 20: Confusion matrix of supervised HCR on test data.

Self-supervised Handwritten Character Recognition Table 7 shows the validation and test accuracies for self-HCR models trained with the MSE, BCE, and perceptual losses. The MSE loss resulted in the highest standard deviation, suggesting it was the least robust to random factors such as weight initialization and training splits. The perceptual loss showed the lowest variability and the highest accuracy on both validation and test sets. However, its performance was moderate compared to the supervised HCR model. These results are reflected in the corresponding loss curves (Figures 21a, 21b, and 21c). For completeness, the loss curve of the supervised HCR model is shown in Figure 19.

To gain more insight into the models' performances, we consider the confusion matrices in Figures 22a, 22b, and 22c of the MSE, BCE, and perceptual losses, respectively. While they show a strong bias towards digit 1 for the MSE loss, and digits 0 and 8 for the BCE loss, no clear bias can be observed for the perceptual loss. However, misclassification of images with the digits 5 and 3 were frequent across all self-HCR models.

Lastly, a small set of 16 synthetic images from the test set with their class predictions are examined to verify the models' output, displayed in Figures 23,

24, and 25 for the MSE, BCE, and perceptual losses, respectively. Notably, in some cases the synthetic images did not accurately reflect the predicted digit. For example, one case from the MSE loss (Figure 23, third and fourth columns) misclassified the digit 0 as 5 with the following predicted probabilities:

$$[0.392, 0.0, 0.0, 0.0, 0.0, 0.404, 0.03, 0.005, 0.169]$$

Here, the index of the probability represents the digit class. The predicted probabilities for the digits 5 and 0 differ with only 0.012, which propagated to the generator. Similarly, a BCE model example (Figure 24, fourth row, last two columns) misclassified a digit 5 as 3, while the synthetic image ambiguously shows a 0 or 3. This, too, is reflected in the predicted class probabilities:

$$[0.399, 0.0, 0.088, 0.5, 0.0, 0.005, 0.005, 0.0, 0.002, 0.0]$$

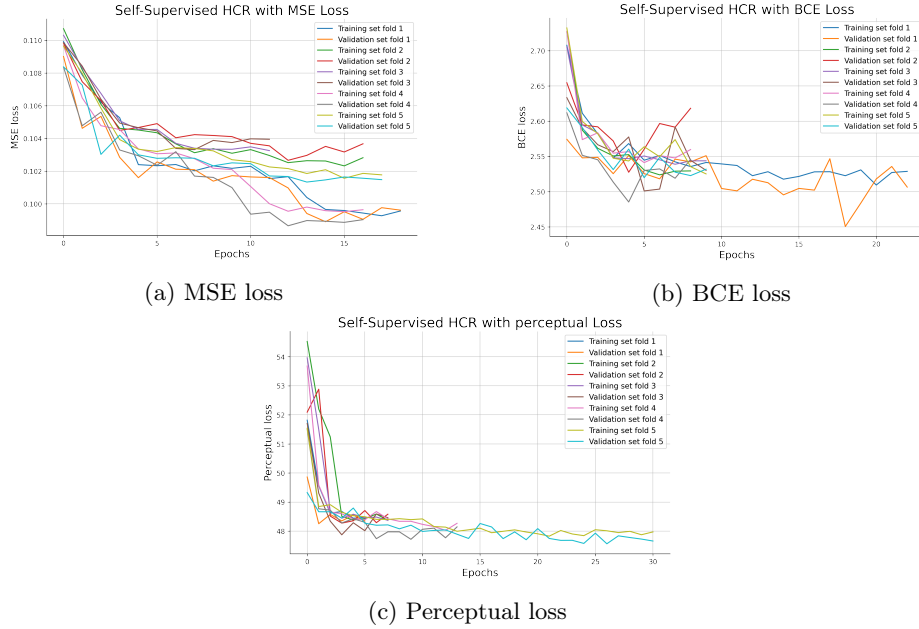


Fig. 21: Loss curves for the self-supervised HCR models trained with the MSE loss (a), BCE loss (b), and perceptual loss (c) using 5-fold cross-validation.

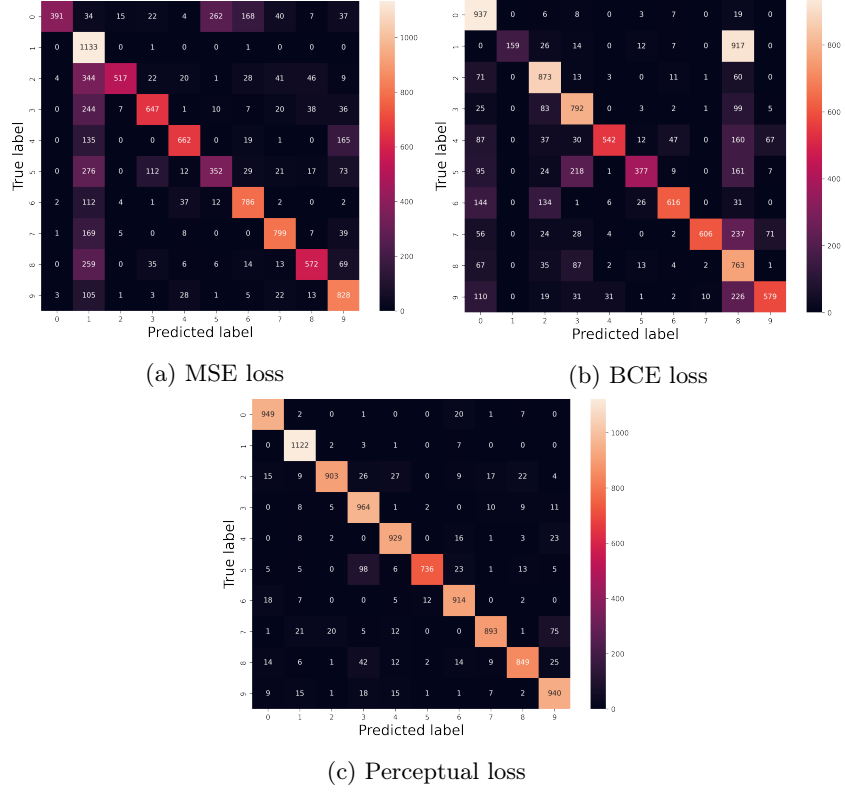


Fig. 22: Confusion matrices on test data for the self-supervised HCR models trained with the MSE loss (a), BCE loss (b), and perceptual loss (c). Test predictions were made by ensembling the HCR models resulting from 5-fold cross-validation.

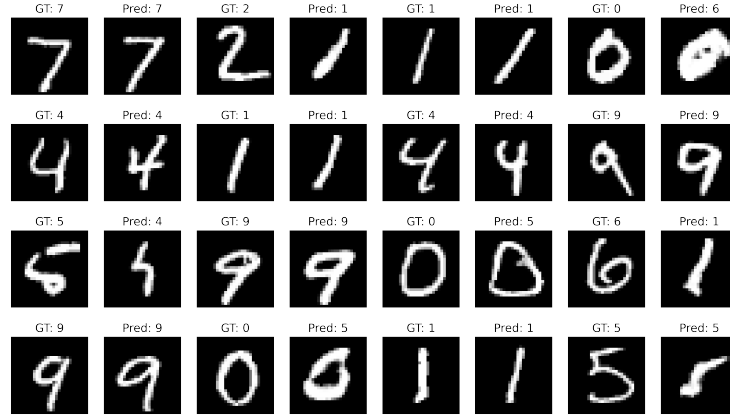


Fig. 23: Input and synthetic image pairs of the self-supervised HCR model trained with the MSE loss. The real images are shown in a column first, and then the synthetic images. GT = ground truth.

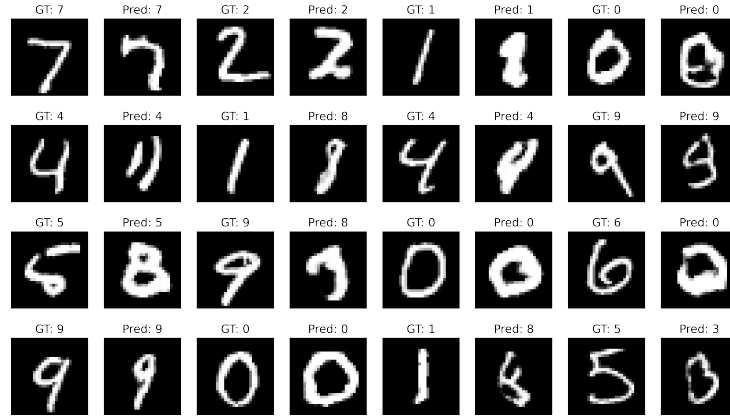


Fig. 24: Input and synthetic image pairs of the self-supervised HCR model trained with the BCE loss. The real images are shown in a column first, and then the synthetic images. GT = ground truth.

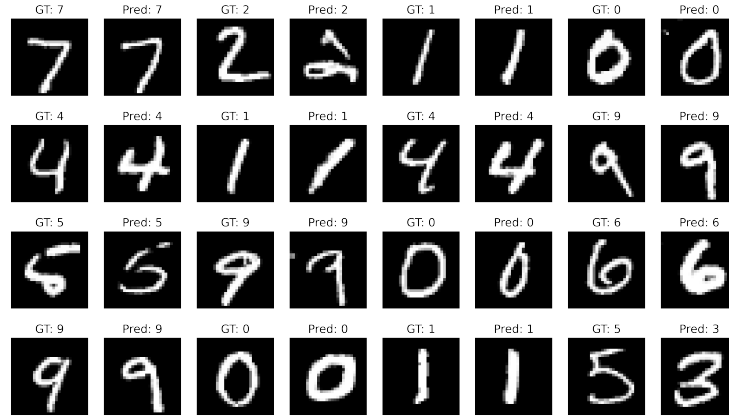


Fig. 25: Input and synthetic image pairs of the self-supervised HCR model trained with the Perceptual loss. The real images are shown in a column first, and then the synthetic images. GT = ground truth.

5.6 Discussion

In this section, we explored the feasibility and implementation of image-based self-supervised HTR by applying the framework to the simpler HCR task with the MNIST dataset. An independent pretrained DCGAN generator was integrated with a simple CNN HCR network by directly inputting HCR-predicted class probabilities to the generator. This was possible as the generator was conditioned on one-hot encoded labels and applying identical image preprocessing to the HCG and self-HCR pipelines.

Inspired by the task of image reconstruction, we experimented with two low-level (MSE and BCE loss) and losses one high-level (perceptual loss) loss to train the self-supervised HCR system. Of these, only the perceptual loss led to a decent performance, with an accuracy of 91.99% on unseen test data, suggesting high-level information is necessary for an effective image-based self-HCR model.

The presented methodology has several points of improvement. For one, model performance could be improved by using a more complex, state-of-the-art HCR architecture. Moreover, using data augmentation and further optimizing hyperparameters size may further enhance performance. However, as the loss computation relies on synthetic image quality, most improvement can likely be made here. This could be done via model selection, with e.g. a more thorough analysis of synthetic image quality throughout all training epochs separately for each model or using a more appropriate evaluation metric, but also by replacing the DCGAN with a better performing HCG architecture.

Self-HCR performance was likely also limited due to the lack of consideration of writing style in generating the synthetic images. This may explain why low-level losses underperformed compared to the perceptual loss. While the perceptual loss does extract higher-level features, it still directly compares the pixels of ex-

tracted feature maps, which are also affected by the writing style. Similarly, the artifacts, specifically more severe character distortions, likely limited the performance too. This highlights the need for a loss function that can robustly capture the differences in character shapes. Given the results and the aforementioned explanations, higher-level loss functions should thus especially be considered for future applications of the self-supervised framework.

References

1. Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G.S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viégas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., Zheng, X.: TensorFlow: Large-scale machine learning on heterogeneous systems (2015), <https://www.tensorflow.org/>, software available from tensorflow.org
2. Barakat, B.K., Alasam, R., El-Sana, J.: Word spotting using convolutional siamese network. In: 2018 13th IAPR International Workshop on Document Analysis Systems (DAS). pp. 229–234 (2018). <https://doi.org/10.1109/DAS.2018.67>
3. Cheng, K., Tahir, R., Eric, L.K., Li, M.: An analysis of generative adversarial networks and variants for image synthesis on MNIST dataset. *Multimedia Tools and Applications* **79**(19) (2020)
4. Chollet, F., et al.: Keras. <https://keras.io> (2015)
5. Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., Hochreiter, S.: GANs trained by a two time-scale update rule converge to a local nash equilibrium. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. p. 6629–6640. NIPS’17, Curran Associates Inc., Red Hook, NY, USA (2017)
6. Johnson, J., Alahi, A., Fei-Fei, L.: Perceptual losses for real-time style transfer and super-resolution (2016)
7. K. He, X. Zhang, S. Ren and J. Sun: Deep residual learning for image recognition. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 770–778 (2016)
8. Khare, N., Thakur, P.S., Khanna, P., Ojha, A.: Analysis of loss functions for image reconstruction using convolutional autoencoder. In: Raman, B., Murala, S., Chowdhury, A., Dhall, A., Goyal, P. (eds.) *Computer Vision and Image Processing*. pp. 338–349. Springer International Publishing, Cham (2022)
9. Lecun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proceedings of the IEEE* **86**, 2278 – 2324 (12 1998)
10. Mirza, M., Osindero, S.: Conditional generative adversarial nets. *arXiv preprint arXiv:1411.1784* (2014)
11. Nikolaidou, K., Retsinas, G., Christlein, V., Seuret, M., Sfikas, G., Smith, E.B., Mokayed, H., Liwicki, M.: WordStylist: styled verbatim handwritten text generation with latent diffusion models. In: *International Conference on Document Analysis and Recognition*. pp. 384–401. Springer (2023)
12. Puigcerver, J.: Are multidimensional recurrent layers really necessary for handwritten text recognition? In: *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*. vol. 01, pp. 67–72 (2017)

13. Retsinas, G., Sfikas, G., Gatos, B., Nikou, C.: Best practices for a handwritten text recognition system. arXiv preprint arXiv:2404.11339 (2024)